HIGH PERFORMANCE COMPUTING

Assginment - 3

Name : M.Harshith

HT NO : 2303A51444

Batch No : 21

Assignment 1: Parallel Vector Computation (Numba Parallel Loop) Scenario A satellite data center processes very large numerical vectors. Serial Python code is too slow for real-time analytics. Objective Implement and parallelize vector operations using Numba parallel loops. Tasks

1. Write a serial Python function: C[i]=α×A[i]+B[i]C[i] = \alpha \times A[i] + B[i]C[i]=α×A[i]+B[i]
1. Convert it to a parallel loop using prange.
2. Measure runtime for increasing vector sizes.
3. Compare serial vs parallel performance. Learning Outcomes Data parallelism in Python Numba vs OpenMP analogy Speedup measurement

Code :

```
import numpy as np
import time
from numba import njit, prange

def serial_vector_compute(A, B, alpha):
    C = np.empty_like(A)
    for i in range(len(A)):
        C[i] = alpha * A[i] + B[i]
    return C

@njit(parallel=True)
def parallel_vector_compute(A, B, alpha):
    C = np.empty_like(A)
    for i in prange(len(A)):
        C[i] = alpha * A[i] + B[i]
    return C
```

Assignment 2: Parallel Matrix Multiplication Scenario

21-01-26 Tuesday A climate modeling application requires large matrix multiplications, making Python loops a bottleneck. Objective Parallelize nested loops using Numba prange. Tasks

1. Implement serial matrix multiplication.
1. Parallelize the outer loop.
2. Parallelize using collapsed loops logic.

3. Analyze cache behavior and performance. Learning Outcomes Nested loop parallelism Memory access patterns Parallel overhead in Python

Code :

```
def serial_matmul(A, B):
    n = A.shape[0]
    C = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i, j] += A[i, k] * B[k, j]
    return C

@njit(parallel=True)
def parallel_matmul_outer(A, B):
    n = A.shape[0]
    C = np.zeros((n, n))
    for i in prange(n):
        for j in range(n):
            for k in range(n):
                C[i, j] += A[i, k] * B[k, j]
    return C

@njit(parallel=True)
def parallel_matmul_collapsed(A, B):
    n = A.shape[0]
    C = np.zeros((n, n))
    for idx in prange(n * n):
        i = idx // n
        j = idx % n
        for k in range(n):
            C[i, j] += A[i, k] * B[k, j]
    return C
```

Assignment 3: Load Balancing with Irregular Workloads Scenario An image processing pipeline processes images with different resolutions, leading to uneven computation time. Objective Study load imbalance in parallel loops. Tasks

1. Simulate variable workload per iteration.
1. Parallelize using prange.
2. Observe execution time variation.
3. Discuss limitations of scheduling in Python. Learning Outcomes Load imbalance Comparison with OpenMP scheduling Practical HPC limitations in Python

Code :

```
@njit
def heavy_work(x):
    s = 0
    for i in range(x):
        s += i * i
```

```
    return s

@njit(parallel=True)
def load_balanced_task(workloads):
    results = np.zeros(len(workloads))
    for i in prange(len(workloads)):
        results[i] = heavy_work(workloads[i])
    return results
```

Assignment 4: Parallel Reduction Operations Scenario A scientific simulation computes global statistics from large datasets. Objective Use parallel reduction patterns. Tasks

1. Compute sum and maximum values.
1. Implement serial and parallel versions.
2. Verify correctness and performance.

Code :

```
def serial_reduction(arr):
    total = 0.0
    maximum = arr[0]
    for x in arr:
        total += x
        if x > maximum:
            maximum = x
    return total, maximum

@njit(parallel=True)
def parallel_reduction(arr):
    total = 0.0
    maximum = arr[0]
    for i in prange(len(arr)):
        total += arr[i]
        if arr[i] > maximum:
            maximum = arr[i]
    return total, maximum
```

Assignment 5: Parallel Monte Carlo Simulation for π Estimation Scenario A computational finance and physics lab uses Monte Carlo simulations that require billions of random samples. Serial execution is too slow, so parallel loop execution is required. Objective

10-01-2026EOD

Use parallel loops to accelerate a Monte Carlo simulation, and analyze scalability similar to OpenMP parallel for. Problem Description Estimate the value of π (pi) using the Monte Carlo method: π≈4×points inside circletotal points\pi \approx 4 \times \frac{\text{points inside circle}}{\text{total points}}π≈4×total pointspoints inside circle Random points are generated inside a unit square. Points falling inside the unit circle are counted. Tasks

1. Implement a serial Monte Carlo simulation.

1. Parallelize the loop using Numba prange.
2. Perform experiments with increasing number of samples.
3. Measure execution time and speedup.
4. Discuss race conditions and reduction handling. Constraints □ Number of samples ≥ 50 million □ Use Numba JIT compilation □ Avoid Python-level random number generation inside loops

Code :

```
import numpy as np
import time
from numba import njit, prange

# ============================================================
# SERIAL MONTE CARLO PI ESTIMATION
# ============================================================
def monte_carlo_pi_serial(x, y):
    inside = 0
    n = len(x)
    for i in range(n):
        if x[i] * x[i] + y[i] * y[i] <= 1.0:
            inside += 1
    return 4.0 * inside / n


# ============================================================
# PARALLEL MONTE CARLO PI ESTIMATION (NUMBA + PRANGE)
# ============================================================
@njit(parallel=True)
def monte_carlo_pi_parallel(x, y):
    inside = 0
    n = len(x)

    # Reduction handled safely by Numba
    for i in prange(n):
        if x[i] * x[i] + y[i] * y[i] <= 1.0:
            inside += 1

    return 4.0 * inside / n


# ============================================================
# EXPERIMENTS WITH INCREASING SAMPLE SIZES
# ============================================================
if __name__== "__main__":

    sample_sizes = [50_000_000, 75_000_000, 100_000_000]

    for N in sample_sizes:
        print("\n==============================================")
```

```python
    print(f"Number of samples: {N}")
    print("==============================================")

    # Generate random points OUTSIDE the loop (constraint satisfied)
    x = np.random.rand(N)
    y = np.random.rand(N)

    # ---------- SERIAL EXECUTION ----------
    start = time.time()
    pi_serial = monte_carlo_pi_serial(x, y)
    serial_time = time.time() - start

    # ---------- PARALLEL EXECUTION ----------
    start = time.time()
    pi_parallel = monte_carlo_pi_parallel(x, y)
    parallel_time = time.time() - start

    # ---------------- RESULTS ----------------
    print(f"Serial π Estimate   : {pi_serial}")
    print(f"Parallel π Estimate : {pi_parallel}")
    print(f"Serial Time (s)     : {serial_time:.4f}")
    print(f"Parallel Time (s)   : {parallel_time:.4f}")
    print(f"Speedup             : {serial_time / parallel_time:.2f}x")
```

OUTPUT :

```
==============================================
Number of samples: 50000000
==============================================
Serial π Estimate   : 3.14173
Parallel π Estimate : 3.14173
Serial Time (s)     : 40.8932
Parallel Time (s)   : 2.3204
Speedup             : 17.62x

==============================================
Number of samples: 75000000
==============================================
Serial π Estimate   : 3.14206464
Parallel π Estimate : 3.14206464
Serial Time (s)     : 60.6180
Parallel Time (s)   : 0.0951
Speedup             : 637.42x

==============================================
Number of samples: 100000000
==============================================
Serial π Estimate  : 3.141673
Parallel π Estimate : 3.141673
Serial Time (s)    : 84.0550
```

```
Parallel Time (s)   : 0.1712
Speedup             : 490.86x
```